

Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 23.Б08-мм

Разработка системы сбора статистики со строительных объектов

Речкин Вадим Сергеевич

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
ст. преподаватель кафедры ИАС, к.ф.-м.н., Азимов Р. Ш.

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор	5
2.1. Обзор существующих решений	5
2.2. Обзор моделей семейства Ultralytics	6
2.3. Обзор алгоритмов трекинга	9
3. Описание решения	11
3.1. Способы получения статистик	11
3.2. Создание ML pipeline для обработки видео	12
3.3. Создание backend-сервиса	13
3.4. Создание статистического отчёта	15
3.5. Описание экспериментов	16
4. Эксперимент	17
4.1. Условия эксперимента	17
4.2. Исследовательские вопросы	17
4.3. Метрики	17
4.4. Результаты	18
4.5. Выводы	20
Заключение	21
Список литературы	22

Введение

Строительная область включает в себя большое количество процессов, каждый из которых требует контроля и анализа результатов. Эффективность работы напрямую зависит от качества контроля. Основной подход к контролю над объектами — это использование специально обученных людей, таких как охранники, прорабы, инженеры и другие специалисты. В данном варианте есть несколько проблем: во-первых, любая ручная работа требует денежных затрат, что для продолжительного проекта может навредить бюджету, во-вторых, невозможность исключить влияние человеческого фактора: сокрытие информации, некачественная работа, воровство и т. д., а в-третьих, результаты контроля хотят получать разные люди, от руководителя до заказчиков объекта.

В настоящее время многие компании стараются автоматизировать процессы и переложить большинство задач программным системам. Развитие технологий, методов машинного обучения и компьютерного зрения позволяет автоматизировать контроль над строительными объектами. Существует много решений, включающих в себя турникеты, умные системы видеонаблюдения, датчики движения и другие подходы. Зачастую такие системы являются дорогостоящими и не все компании могут себе позволить их внедрение.

Возникает потребность в открытых умных системах для контроля над объектами, внедрение которых будет недорогим, а результаты контроля будут доступны для заинтересованных лиц. Большим плюсом для таких систем является возможность их интеграции в существующее оборудование, возможность модернизации и расширения функционала.

В данной работе была разработана открытая система сбора статистики со строительных объектов. В качестве источника данных использовался RTSP [8]-стрим с камеры видеонаблюдения, охватывающей зону входа и выхода, после чего осуществлялись обработка данных, формирование статистического отчёта и его рассылка через мессенджер.

1. Постановка задачи

Целью работы является разработка системы сбора статистики со строительных объектов. Для её выполнения были поставлены следующие задачи:

1. Провести обзор существующих решений.
2. Разработать и реализовать подход записи и обработки данных со строительных объектов.
3. Спроектировать и реализовать ML-pipeline для обработки видео.
4. Разработать телеграм-бота для рассылки отчётов со статистикой.
5. Настроить генерацию отчёта со статистикой.
6. Измерить метрики качества работы системы и провести валидацию результатов.

2. Обзор

В этом разделе будут описаны существующие решения, разобраны модели, используемые в работе, а также алгоритмы отслеживания траекторий на видео.

2.1. Обзор существующих решений

Идея данной работы не уникальна. Существует множество реализаций, предназначенных для разных целей. Большинство сложных и качественных систем делается на заказ для компаний, поэтому к ним нет открытого доступа. Ниже будет сравнение нескольких открытых реализаций, решающих похожую задачу. Все работы ниже достойны внимания, но каждую реализацию стоит выбирать конкретно под свой проект.

- **Equipment Utilization and Activity Classification¹**: система для анализа видео, которая способна классифицировать действия строительной техники на объекте, а именно: простой, загрузку и разгрузку, движение и ожидание. После этого система собирает статистику, которая доступна в пользовательском интерфейсе. В этой работе реализована важная для многих компаний система, подсчитывающая время простоя каждой единицы техники, но есть несколько отличий от текущей работы: во-первых, это лишь модуль с обработкой, он принимает путь к сохранённому файлу и не делает рассылку, только обработку, из-за этого у пользователя могут возникнуть трудности в развёртывании, во-вторых, система не отслеживает статистику по людям, что также важно для компаний, а в-третьих, для качественной работы системы требуется камера с хорошим разрешением и без слепых зон, что часто является большой проблемой на строительных объектах.

¹Ссылка на репозиторий с работой: <https://github.com/samir-m0hamed/Equipment-Utilization-and-Activity-Classification>.

- **Worker Safety Twin**²: система, способная распознавать людей и отображать их в 3D-объекте, что также позволяет отслеживать траектории рабочих в течение дня. Этот проект частично похож на задачу, которую будет решать предложенная в этой работе система, но цель этой работы — отрисовать траектории работников для дальнейшего анализа, а не анализировать ситуацию в общем без привязки к конкретным людям, что делает требования к видеокамерам значительно выше, а также требует их большего количества.
- **SiteWatch**³: Этот проект больше про безопасность на объекте: он анализирует видео и проверяет наличие касок, находит случаи, когда возникают опасные ситуации, но не собирает статистику о работе.

Существует действительно много реализаций похожих задач, которые анализируют видео и собирают статистику, но все они используются в разных сценариях. В данной работе будет создана система именно для отслеживания количества персонала и техники на строительном объекте в течение дня.

2.2. Обзор моделей семейства Ultralytics

В работе использовались две модели из семейства Ultralytics ⁴: YOLO [9] и RT-DETR [3]. Обе модели изначально обучены для решения задачи детекции, но их также можно использовать для других задач.

2.2.1. Обзор модели YOLO

Модель YOLO (You Only Look Once) на момент выхода была революционной, так как до этого задачу детекции решали другие модели,

²Ссылка на репозиторий с работой: https://github.com/almosenja/Worker-Safety-Twin?utm_source=chatgpt.com.

³Ссылка на репозиторий с работой: https://github.com/AitchEm-bot/sitewatch?utm_source=chatgpt.com.

⁴Документация Ultralytics: <https://docs.ultralytics.com/> (дата обращения: 25 сентября 2026 г.).

например, модели семейства R-CNN [7]. Они решали задачу посредством поиска кандидатов, а потом из каждого кандидата извлекали признаки и делали классификацию. Такой подход называется двустадийным, и он был медленным. YOLO же решила перейти к одностадийному варианту, где рамки для объектов предсказывались сразу. Это ускорило решение задачи детекции в десятки раз, а точность такой детекции была немного ниже, но в то время это считалось отличным результатом.

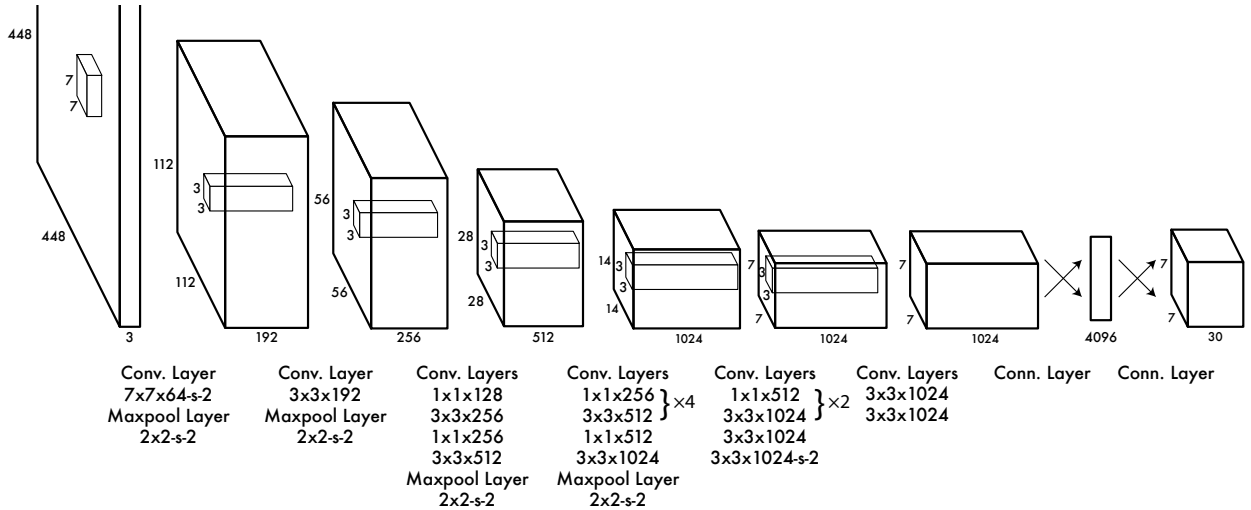


Рис. 1: Архитектура YOLO

За основу была взята архитектура GoogLeNet [5], представляющая собой обычную свёрточную сеть, в которой были заменены последние слои для предсказания рамок, вероятностей класса и уверенности модели в том, что в рамке находится объект. После этого специальным алгоритмом Non-Maximum Suppression отбирались лучшие кандидаты.

В настоящее время модели YOLO остаются в списке лучших для решения задачи детекции и часто являются отличным **baseline**, а после дообучения и других модификаций остаются в итоговом результате.

2.2.2. Обзор модели RT-DETR

Модель RT-DETR (Real-Time DETection Transformer) — версия модели DETR, предназначенная для работы в режиме **real-time**. После выхода ViT [6] (Vision Transformer) — трансформерной модели для изображений — создание трансформер-модели для задачи детекции было

лишь вопрос времени. Трансформерная архитектура показывала SOTA результаты практически во всех задачах, поэтому её старались применить и для других задач.

Архитектура DETR [4] использует в **encoder** части идею ViT, которая кодирует участки изображения в векторы. Затем вычисляется **self-attention** между обучаемыми векторами запросов, а затем — **cross-attention** между выходом **self-attention** и векторами изображения. После N таких слоёв применяются полносвязные слои и выполняется предсказание.

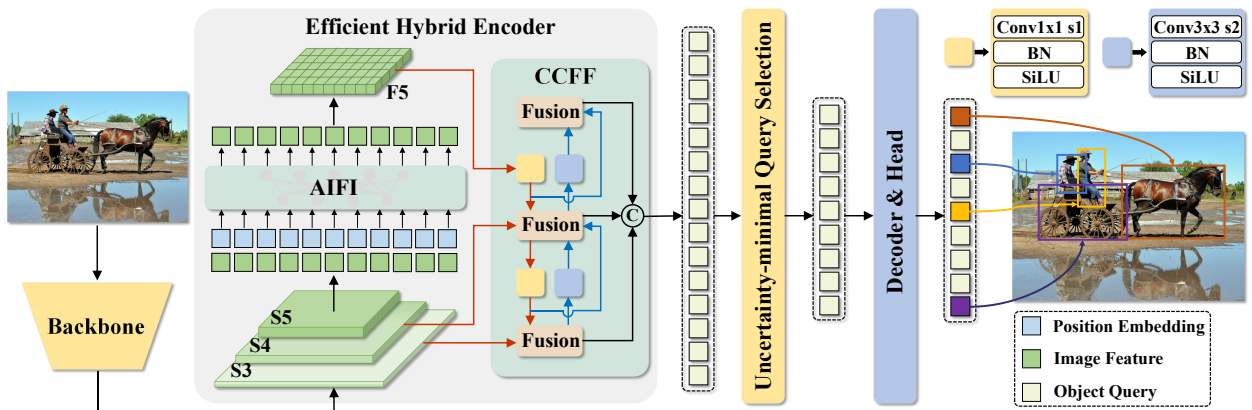


Рис. 2: Архитектура RT-DETR

В версии **real-time** архитектура немного изменена. В ней используется **Efficient Hybrid Encoder**, который позволяет применять различные способы агрегации векторов на разных уровнях детализации, вместо применения **self-attention** для каждого вектора. Для объединения мелких деталей используется **self-attention**, а для средних — свёртки 1×1 .

Также в версии **real-time** модифицирован **decoder**: вместо выполнения сложных вычислений **self-attention** и **cross-attention** для каждого обученного вектора-запроса после извлечения признаков рассчитываются предварительные предсказания, затем отбираются только уверенные предсказания. Есть ещё несколько модификаций, которые ускоряют DETR, но о них можно прочитать в статье.

Такая модель зачастую может превосходить по точности модель YOLO, но имеет свои минусы: большее время работы даже для версии

`real-time`, большие требования к железу, а также трансформер-модели сложнее качественно дообучить, по сравнению со свёрточными.

2.3. Обзор алгоритмов трекинга

Задача детекции заключается в поиске объектов на изображении, но часто возникает необходимость отследить перемещение объекта. Это задача трекинга. Алгоритмы трекинга часто работают уже с предсказаниями, полученными моделью детекции, анализируя их. Простой анализ результатов детекции не даст нужных результатов: специальные алгоритмы трекинга предсказывают направления движения, сопоставляя их с предсказаниями модели, умеют работать с перекрытиями и т.д. Ниже будут рассмотрены примеры алгоритмов трекинга, которые упоминаются в работе.

2.3.1. Обзор алгоритма ByteTrack

ByteTrack [2] — алгоритм для отслеживания объектов на видео, который работает с предсказаниями детекции без использования внутренней эмбедин-модели. Этот алгоритм не является новым, но показывает хорошие результаты, и его можно использовать в качестве хорошего `baseline`.

Принцип его работы достаточно прост: ByteTrack использует результаты детектора, разделяя рамки на группы с высокой и низкой уверенностью.

Такой подход хорошо работает в случаях, когда допустима периодическая потеря объекта, а требуется лишь определять примерные траектории людей без привязки к конкретным экземплярам.

2.3.2. Обзор алгоритма BoT-SORT

Алгоритм BoT-SORT [1] берёт за основу ByteTrack и улучшает его, модернизируя проверки. BoT-SORT может работать с камерами, у которых положение может меняться, например, на видеорегистраторе автомобиля. Также он улучшает часть с предсказанием положения объектов,

учитывая уверенность модели детекции, выполнившей предсказания. Ещё одно важное улучшение — добавление Re-ID: трекер использует внутреннюю эмбединг-модель, чтобы получать дополнительные признаки рамки. Технология Re-ID применялась и в других алгоритмах трекинга, но раньше была проблема, что на неё сильно полагались при предсказании, в BoT-SORT это учли и скомбинировали с технологиями ByteTrack.

3. Описание решения

В данном разделе рассмотрены различные подходы к решению задачи мониторинга, описаны алгоритм обработки видеозаписей и процесс создания единого сервиса с Telegram-ботом для рассылки, а также представлена инструкция по обучению модели на данных конкретного объекта.

3.1. Способы получения статистик

В данной работе система была рассчитана на использование одной камеры на объекте, поэтому возникали ограничения, накладываемые на проект. К примеру, нельзя просто подсчитывать количество всех людей и машин в кадре, находившихся в зоне строительного объекта, так как на объекте могли быть слепые зоны или они могли появиться в процессе строительства. Также, поскольку система была рассчитана на экономное подключение, могли использоваться стандартные камеры с плохим разрешением, что ухудшало предсказания детекции.

Было решено отслеживать определённую зону, которую рабочие и техника проходили как при входе, так и при выходе. Чаще всего это зона ворот или аналогичная часть объекта. Данный подход имел несколько преимуществ:

1. Зона ворот на камере всегда должна просматриваться, так как иначе служба охраны не сможет качественно выполнять свою работу. Из этого следует, что слепые зоны не мешали при таком подходе. Поэтому просматриваемость ворот для данной работы считается требованием.
2. Даже при низком качестве камеры после дообучения модели и при грамотной настройке трекера системе было достаточно один раз обнаружить объект, а трекер мог самостоятельно предсказывать положение объекта, что повышало устойчивость.

У данного подхода также был серьёзный недостаток: если ворот на

объекте несколько или существуют иные способы уйти с объекта или зайти на него, данный подход не сможет корректно посчитать статистику. Решением этой проблемы может стать подключение второй камеры или подача второй зоны ворот из первого видео, но тогда решение будет требовать небольших доработок.

3.2. Создание ML pipeline для обработки видео

Далее описано, как работал ML pipeline в данной работе. Он был представлен отдельным сервисом, который принимал заявки на обработку видео и возвращал файл для подготовки статистического отчёта.

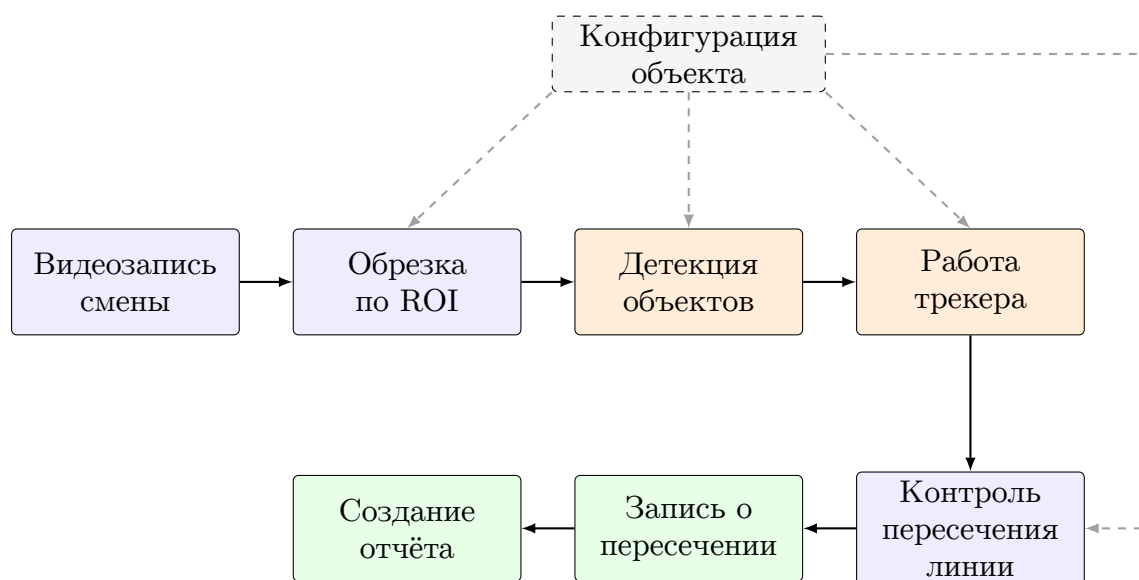


Рис. 3: ML-пайплайн обработки видеозаписи строительного объекта.

Для хранения конфигурации объектов использовалась PostgreSQL⁵, которая являлась удобной СУБД и также использовалась далее в работе.

Для повышения качества модель была обучена на обрезанных изображениях с зоной ворот, так как объекты были крупнее, чем на исходном изображении. В качестве модели детекции была выбрана именно модель семейства YOLO. Она обеспечила обработку видео в режиме

⁵Документация PostgreSQL: <https://www.postgresql.org/docs/> (дата обращения: 25 сентября 2026 г.).

`real-time`, имела хорошее качество детекции и удобный интерфейс обучения и инференса. В качестве аналога можно взять RT-DETR. RT-DETR из-за своей архитектуры более требовательна и имеет меньшую скорость работы по сравнению с YOLO, но может показать более высокие метрики в некоторых случаях.

Далее модель передавала результаты детекции трекеру. Решено было взять ByteTrack, так как его можно использовать через библиотеку Supervision⁶, на которой была реализована большая часть ML pipeline. Также он значительно проще в настройке по сравнению с другими трекерами из этой же библиотеки. Для более качественного результата рекомендуется использовать BoT-SORT: его также можно использовать через Supervision, но главное преимущество — он содержит эмбединг-модель, что делает его более точным и устойчивым, но сложным к настройке.

Вход и выход работников и техники на объект отслеживались через пересечение заданной линии. Такой функционал есть в библиотеке Supervision. При пересечении линии объектом в статистику добавлялась соответствующая запись.

Итоговый подсчёт статистики выполнялся отдельно после обработки всей смены: рассчитывались показатели и формировались графики. Недостаток данного метода заключается в том, что в течение дня одни и те же сотрудники могли выходить и заходить на объект, но при учёте этого в подсчёте статистики проблема не возникала.

3.3. Создание backend-сервиса

Архитектура проекта разделена на два сервиса: один из них — описанный выше ML-сервис, а другой — backend-сервис. В этом разделе будет подробно описана именно часть, связанная с backend.

⁶Документация библиотеки Supervision: <https://supervision.roboflow.com/> (дата обращения: 25 сентября 2026 г.).

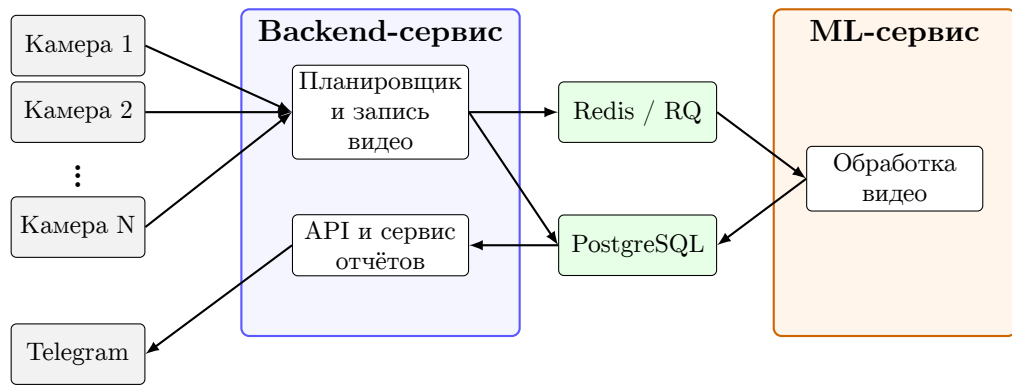


Рис. 4: Взаимодействие backend- и ML-сервисов при параллельной записи видеопотоков с камер строительного объекта.

Для сбора данных backend-сервис по расписанию подключался к RTSP-стримам и сохранял видеозаписи в хранилище. Для фиксации дальнейших действий в базе данных PostgreSQL создавалась запись с идентификатором объекта, временем начала и окончания смены, статусом записи и путём к сохранённому видео.

После завершения записи backend создавал задачу обработки и помещал её в очередь Redis ⁷ с использованием RQ ⁸. Такой подход позволил записывать несколько видеозаписей одновременно и обрабатывать их все одним ML-сервисом. Также эту реализацию можно улучшить, например, добавив ещё ML-сервисы, которые будут брать задачи на обработку, или при наличии разных моделей добавить параметр, который позволит ожидать определённый ML-сервис.

ML-сервис описан выше: он брал задачу из очереди, подгружал параметры и выполнял обработку, после чего заносил в PostgreSQL информацию о результате.

После успешной обработки всех смен за день backend-сервис получал результаты из PostgreSQL, объединял их в итоговую таблицу и формировал отчёт. Подготовленный файл вместе с текстовой сводкой и графиками отправлялся подписанным получателям через Telegram-бота ⁹.

⁷Документация Redis: <https://redis.io/docs/latest/> (дата обращения: 25 сентября 2026 г.).

⁸Документация RQ: <https://python-rq.org/docs/> (дата обращения: 25 сентября 2026 г.).

⁹Документация Telegram Bot API: <https://core.telegram.org/bots/api> (дата обращения: 25 сентября 2026 г.).

Telegram-бот реализован для отправки сообщений пользователям, которые хотя бы раз написали ему. Этот базовый функционал позволяет расширить бота и адаптировать его к дополнительным требованиям.

3.4. Создание статистического отчёта

Статистический отчёт должен быть небольшим, информативным и не вводящим в заблуждение. Выше уже была описана проблема: в предложенном подходе работники и техника, которые выходят и заходят на объект, могли дополнительно увеличивать счётчик количества заходов и выходов, поэтому использовать эти величины в качестве статистики — плохая идея.

Для данного проекта одним из наиболее информативных показателей являлось среднее количество работников и техники в течение рабочего дня. Также стоит отметить, что рассчитывать статистику за полный рабочий день не имеет большого смысла: ночью количество персонала будет значительно ниже, чем днём, поэтому логичнее разбивать видео со смены на несколько частей, соответствующих времени работы смен работников.

$$\overline{N} = \frac{1}{T_{\text{см}}} \int_{\tau_0}^{\tau_{m+1}} N(t) dt \approx \frac{1}{T_{\text{см}}} \sum_{i=0}^m N_i (\tau_{i+1} - \tau_i), \quad T_{\text{см}} = \tau_{m+1} - \tau_0.$$

где

- $\overline{N}$ — среднее количество людей за смену;
- $T_{\text{см}}$ — длительность смены;
- $N(t)$ — количество людей на объекте в момент времени t ;
- τ_0 и τ_{m+1} — время начала и окончания смены соответственно;
- τ_i — время i -й записи о пересечении линии;
- N_i — количество людей в начале интервала $[\tau_i, \tau_{i+1})$;

- m — количество записей о пересечении линии за смену.

Дополнительно к среднему значению в этом проекте был построен график зависимости среднего значения за определённый час от времени. Это позволило при желании уточнить, когда прибывала техника и когда рабочие находились на объекте.

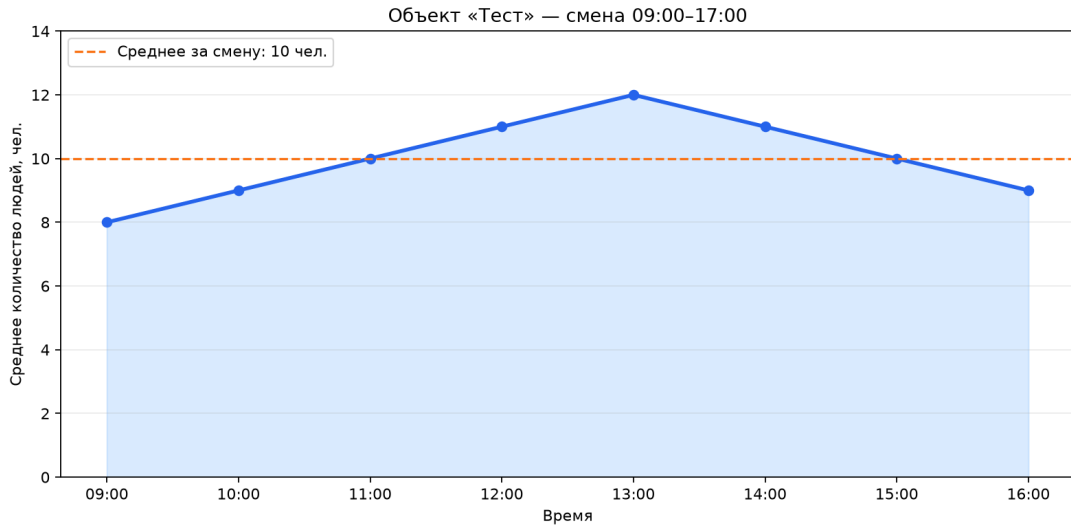


Рис. 5: Пример графика статистического отчёта за смену

3.5. Описание экспериментов

В этой работе были проведены эксперименты по обучению модели под домен строительных объектов, а также эксперимент по замеру end-to-end качества сервиса.

Для тонкой настройки модели YOLO использовался датасет MOCS¹⁰. Он содержит разные виды техники и рабочих, что соответствует доменной области.

Для измерения end-to-end-качества использовался датасет CMOT¹¹. Он содержит записи со строительных объектов, где отмечены траектории рабочих и строительной техники.

¹⁰Датасет MOCS: <https://www.kaggle.com/datasets/xiaopan9802/mocs-dataset> (дата обращения: 28 сентября 2026 г.).

¹¹Датасет CMOT: <https://github.com/XZ-YAN/CMOT-Dataset> (дата обращения: 28 сентября 2026 г.).

4. Эксперимент

В данном разделе описаны эксперименты по обучению модели, замеру метрик детекции и end-to-end метрик.

4.1. Условия эксперимента

Эксперимент по обучению модели YOLO проводился на платформе kaggle. Технические характеристики указаны ниже:

- CPU: Intel Xeon 2.00 GHz
- CPU count: 4
- GPU: NVIDIA Tesla T4 16 GB
- GPU count: 2
- RAM: 31.35 GB

4.2. Исследовательские вопросы

В ходе экспериментов нужно было ответить на следующие исследовательские вопросы.

RQ1 : позволяет ли предложенный подход получить качественные end-to-end-метрики?

4.3. Метрики

В работе использовались известные метрики для задачи детекции: mAP50, mAP50-90, precision, recall, F1. Все перечисленные метрики применяются для оценки качества детекции.

В качестве end-to-end-метрики использовалась пользовательская метрика доли пропущенных строк. Для работников и техники MAPE вычислялась как абсолютная относительная ошибка среднего количества объектов:

$$\text{MAPE} = \frac{|\bar{x}_{\text{исх.}} - \bar{x}_{\text{пред.}}|}{\bar{x}_{\text{исх.}}} \cdot 100\%.$$

4.4. Результаты

4.4.1. Результаты обучения модели YOLO

Значения гиперпараметров и других настроек для обучения модели находятся в файле `/models/yolo_training.ipynb`.

Таблица 1: Метрики модели YOLO после дообучения

Precision	Recall	mAP50	mAP50-90
0.879	0.720	0.812	0.601

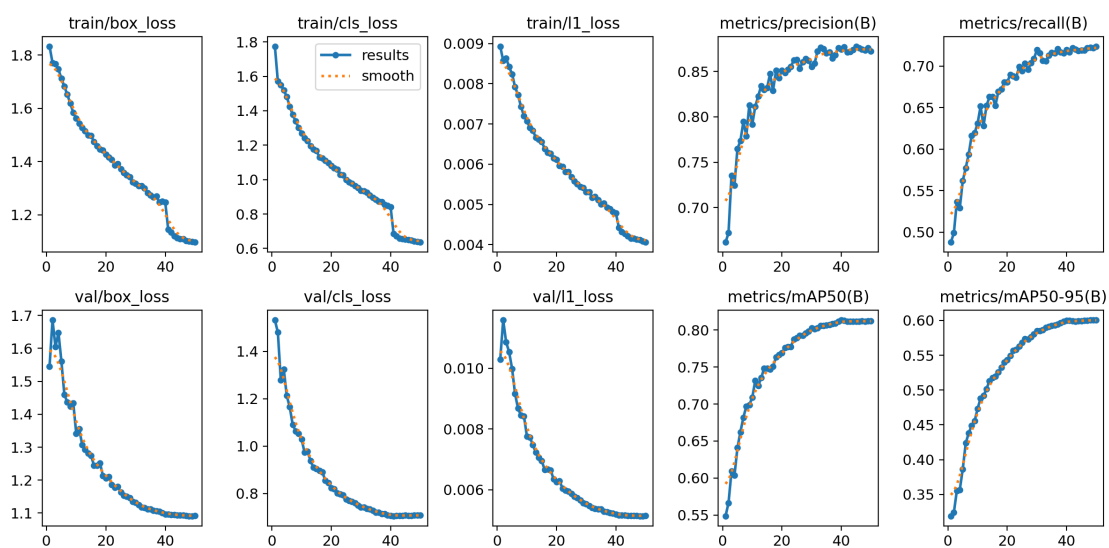


Рис. 6: Динамика функции потерь при обучении YOLO и значения метрик

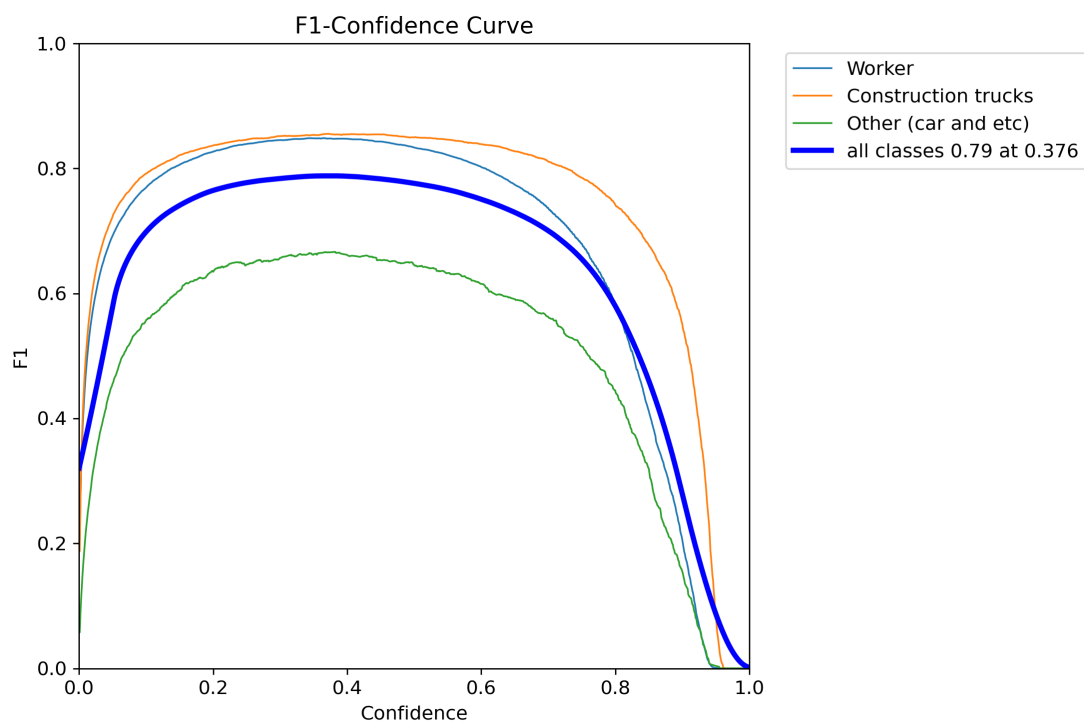


Рис. 7: Зависимость F1-меры от порога уверенности модели YOLO

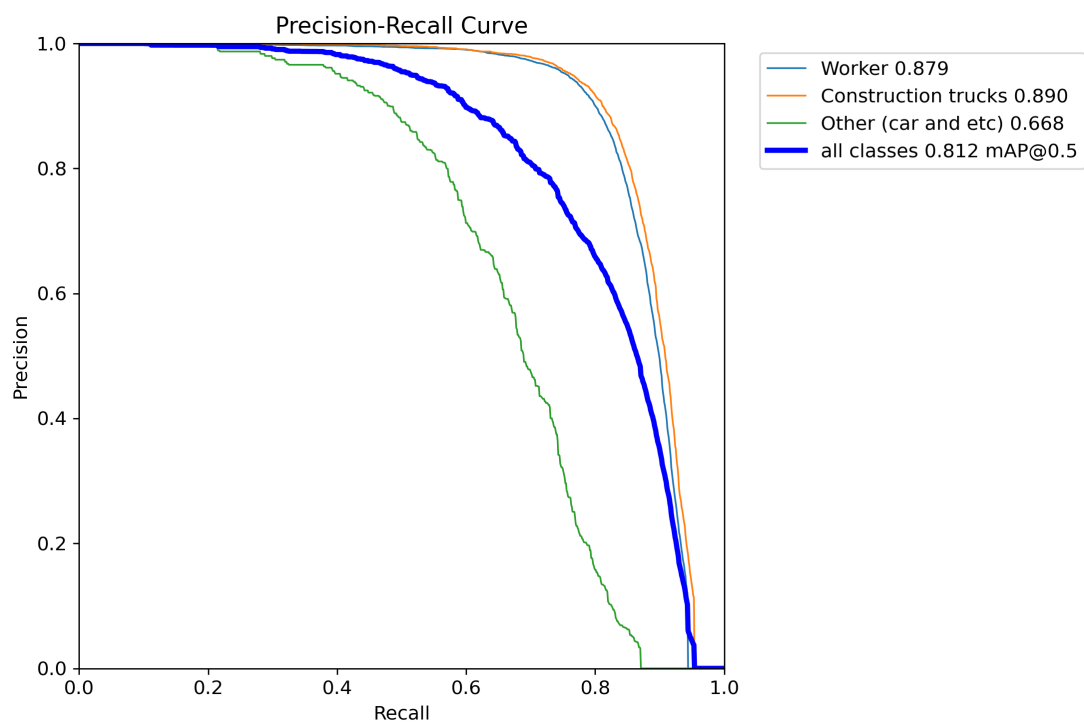


Рис. 8: Кривая precision-recall модели YOLO после дообучения

4.4.2. Результаты замера end-to-end метрик

Эксперимент проводился на трёх тестовых видео; исходные и предсказанные данные находятся в репозитории в директории `metrics`.

Таблица 2: Результаты замера end-to-end метрик

Смена	Работники исх./пред.	Техника исх./пред.	Доля пропусков, %	MAPE работников, %	MAPE техники, %
0001	9.26 / 9.23	0.85 / 0.85	4.38	0.32	0.01
0002	12.32 / 12.30	1.63 / 1.57	5.16	0.21	3.62
0003	6.11 / 5.68	2.12 / 2.12	4.72	7.04	0.00

4.4.3. RQ1

Из таблицы 2 видно, что полученные end-to-end-метрики имеют приемлемые значения. Это подтверждает возможность качественной работы сервиса.

4.5. Выводы

На данных строительной области модель может быть успешно обучена при использовании качественного набора данных. Значения end-to-end-метрик приемлемы, но требуют дальнейшего улучшения. Для каждого домена данных модель можно дообучить, и значения метрик станут лучше.

Заключение

В данной работе был создан ML-сервис для сбора статистики со строительных объектов, способный собирать статистику на основе анализа видео, в котором видна зона ворот. Также в этом сервисе присутствует Telegram-бот, который отправляет отчёты подписанным пользователям. В процессе работы были выполнены поставленные задачи.

1. Был проведён обзор существующих решений (подробнее в разделе 2).
2. Разработан способ сбора статистики и последующей обработки видео (подробнее в подразделе 3.1).
3. Был реализован ML-алгоритм, обрабатывающий данные с помощью дообученной модели YOLO (подробнее в подразделах 3.2 и 4.4.1).
4. Разработан Telegram-бот для ежедневной рассылки результатов обработки (подробнее в подразделе 3.3).
5. Разработан формат статистического отчёта для рассылки (подробнее в подразделе 3.4).
6. Получены метрики обучения модели и работы end-to-end сервиса. Реализация может использоваться для дальнейших проектов (подробнее в подразделе 4.4).

Исходный код данной работы: <https://github.com/VadosikRRR/ObjectStatistick>

Список литературы

- [1] Aharon Nir, Orfaig Roy, Bobrovsky Ben-Zion. BoT-SORT: Robust Associations Multi-Pedestrian Tracking // arXiv preprint arXiv:2206.14651. — 2022. — URL: <https://arxiv.org/abs/2206.14651>.
- [2] ByteTrack: Multi-Object Tracking by Associating Every Detection Box / Yifu Zhang, Peize Sun, Yi Jiang et al. // Computer Vision – ECCV 2022. — 2022. — P. 1–21. — URL: https://doi.org/10.1007/978-3-031-20047-2_1.
- [3] DETRs Beat YOLOs on Real-time Object Detection / Yian Zhao, Wenyu Lv, Shangliang Xu et al. // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. — 2024. — URL: <https://doi.org/10.1109/CVPR52733.2024.01605>.
- [4] End-to-End Object Detection with Transformers / Nicolas Carion, Francisco Massa, Gabriel Synnaeve et al. // Computer Vision – ECCV 2020. — 2020. — P. 213–229. — URL: https://doi.org/10.1007/978-3-030-58452-8_13.
- [5] Going Deeper with Convolutions / Christian Szegedy, Wei Liu, Yangqing Jia et al. // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — 2015. — P. 1–9. — URL: <https://doi.org/10.1109/CVPR.2015.7298594>.
- [6] An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale / Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov et al. // International Conference on Learning Representations. — 2021. — URL: <https://openreview.net/forum?id=YicbFdNTTy>.
- [7] Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation / Ross Girshick, Jeff Donahue, Trevor Darrell, Jitendra Malik // Proceedings of the IEEE Conference on Computer Vi-

sion and Pattern Recognition. — 2014. — P. 580–587. — URL: <https://doi.org/10.1109/CVPR.2014.81>.

- [8] [Real Time Streaming Protocol \(RTSP\)](#) : RFC : 2326 / Internet Engineering Task Force ; Executor: Henning Schulzrinne, Anup Rao, Robert Langhans : 1998. — URL: <https://www.rfc-editor.org/rfc/rfc2326>.
- [9] [You Only Look Once: Unified, Real-Time Object Detection](#) / Joseph Redmon, Santosh Divvala, Ross Girshick, Ali Farhadi // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. — 2016. — P. 779–788. — URL: <https://doi.org/10.1109/CVPR.2016.91>.